TRANSPORT AND TELECOMMUNICATION INSTITUTE

ENGINEERING FACULTY

Laboratory work N1 Part 2

Course: **Programming Languages Concepts**

Theme: **Comparison of AI Tools for Writing Code**

Student: Igors Oļeiņikovs

Student code: 93642

Group: 4501BTA

Riga
2026

CONTENTS

1. LABORATORY WORK TASK 3
2. AI TOOLS USED 3
3. INITIAL TASK 3
4. PROMPT USED 4
5. RESULTS OBTAINED 5
6. COMPARISON TABLE 15
7. ANALYSIS OF RESULTS 16
8. CONCLUSION 17

1. LABORATORY WORK TASK

Objective of the Lab

The objective of this laboratory work is to study the capabilities of modern AI tools for code generation, compare their strengths and weaknesses, and evaluate how useful they are for a beginner programmer. The comparison is based on the same practical task given to three browser-based tools: ChatGPT, Claude, and Gemini.

2. AI TOOLS USED

ChatGPT

OpenAI conversational AI tool with strong support for code generation, explanation, and debugging.

Claude

Anthropic AI assistant known for structured explanations, long-form reasoning, and readable answers.

Gemini

Google AI assistant integrated into a browser-friendly workflow with concise answers and code generation support.

3. INITIAL TASK

Selected programming language: **Python**

Program name: **Number Check**

- The program asks the user to input a number.
- The program determines whether the number is even or odd.
- The program determines whether the number is positive, negative, or zero.
- The result is displayed on the screen.

Python was selected because it is beginner-friendly, concise, and all three AI tools commonly generate Python code correctly for simple educational tasks.

4. PROMPT USED

Unified prompt for all three tools

Write a program in Python that asks the user to enter a number, determines whether the number is even or odd, and whether it is positive, negative, or zero. The program should display the result clearly. Please include comments in the code and explain how the program works in simple terms suitable for a beginner.

Additional improvement prompt

Improve the program so that it handles invalid input (for example, if the user enters text instead of a number). Explain what was changed and why.

5. RESULTS OBTAINED

Each AI tool received the same initial prompt and the same follow-up improvement prompt. The main comparison criteria were: understanding of the task, code correctness, clarity of explanation, usability, and the quality of the improved version with input validation.

The responses showed visibly different coding styles and variable names. ChatGPT used names such as number, sign, and parity. Claude organized the solution into helper functions such as check_even_or_odd and check_sign. Gemini used names such as user_number and user_input.

<u>ChatGPT</u>

Initial response + improved response

ChatGPT answered the task directly, included comments in the code, and gave a clear beginner friendly explanation. In the response it also handled a useful nuance: even/odd only applies to integers, so decimal numbers are reported as neither even nor odd.

Initial generated code from response

```
 # This program asks the user to enter a number
# Then it checks whether the number is even or odd
# And whether it is positive, negative, or zero
# Ask the user to input a number
number = float(input("Enter a number: "))
# Check if the number is positive, negative, or zero
if number > 0:
  sign = "positive"
elif number < 0:
  sign = "negative"
else:
  sign = "zero"
```

```
    # Check if the number is even or odd
    # Note: Even/odd only makes sense for integers
    if number.is_integer():  # Check if the number is a whole
number
      if int(number) % 2 == 0:
        parity = "even"
      else:
        parity = "odd"
    else:
      parity = "neither even nor odd (not an integer)"
    # Display the result
    print("\nResult:")
    print("The number is", sign)
    print("The number is", parity)
```

Brief description of the initial result

- The answer fulfilled the task and included comments plus a simple explanation.
- The code was short, readable, and explicit about integer versus decimal input.
- The variable names in the saved response were number, sign, and parity.

Improved generated code from response

```
     # This program asks the user to enter a number
    # It checks if the input is valid and keeps asking until it
gets a correct number
    # Then it determines whether the number is even/odd and
positive/negative/zero
    while True:
      try:
        # Try to convert user input into a number
        number = float(input("Enter a number: "))
        break  # Exit the loop if input is valid
```

```
      except ValueError:
        # This runs if the user enters something that is not a
number
        print("Invalid input! Please enter a valid number.\n")
    # Check if the number is positive, negative, or zero
    if number > 0:
      sign = "positive"
    elif number < 0:
      sign = "negative"
    else:
      sign = "zero"
    # Check if the number is even or odd
    if number.is_integer():  # Only works for whole numbers
      if int(number) % 2 == 0:
        parity = "even"
      else:
        parity = "odd"
    else:
      parity = "neither even nor odd (not an integer)"
    # Display the result
    print("\nResult:")
    print("The number is", sign)
    print("The number is", parity)
```

Observation after improvement

ChatGPT improved the program in a direct and assignment-focused way: it added a while True loop and try/except ValueError without making the solution much more complicated.

<u>Claude</u>

Initial response + improved response

Claude produced the most structured initial response. It used helper functions, a main() function, formatted output, and it already included invalid-input handling in the first answer, even before the improvement prompt was given.

Initial generated code from response

```
     def check_even_or_odd(number):
      """
      Checks whether a number is even or odd.
      A number is even if dividing it by 2 leaves no remainder.
      The modulo operator (%) gives us the remainder.
      """
      if number % 2 == 0:
        return "Even"
      else:
        return "Odd"
    def check_sign(number):
      """
      Checks whether a number is positive, negative, or zero.
      """
      if number > 0:
        return "Positive"
      elif number < 0:
        return "Negative"
      else:
        return "Zero"
    def main():
      # --- Step 1: Ask the user to enter a number ---
      print("=" * 40)
      print("       Welcome to Number Checker!")
      print("=" * 40)
      # We use a loop so the program can recover if the user types
something invalid
      while True:
```

```
        user_input = input("\nPlease enter a number: ")
        # --- Step 2: Try to convert the input to a number ---
        try:
          number = int(user_input)
          break
        except ValueError:
          print("⚠  That doesn't look like a whole number. Please
try again.")
      # --- Step 3: Determine even/odd and sign ---
      even_odd_result = check_even_or_odd(number)
      sign_result = check_sign(number)
      # --- Step 4: Display the results clearly ---
      print("\n" + "=" * 40)
      print(f"  Results for the number: {number}")
      print("=" * 40)
      print(f"  ➤ Even or Odd : {even_odd_result}")
      print(f"  ➤ Sign        : {sign_result}")
      print("=" * 40)
      if number == 0:
        print("  ℹ  Note: Zero is even, but has no sign.")
        print("=" * 40)
    if __name__ == "__main__":
      main()
```

Brief description of the initial result

- The initial code was already more advanced than required and included input validation before the second prompt.
- The answer emphasized structure, decomposition into functions, and formatted output.
- The explanation was thorough and very educational, but also noticeably longer than the other responses.

Improved generated code from response

```
     MAX_ALLOWED = 1_000_000_000_000  # 1 trillion
    def get_number_from_user():
      while True:
        raw_input = input("\nEnter a number (or 'q' to quit): ")
        if raw_input.strip().lower() == 'q':
          return None
        if raw_input.strip() == "":
          print("  ⚠  You didn't enter anything. Please type a
number.")
          continue
        try:
          number = float(raw_input.strip())
        except ValueError:
          print(f'  ⚠  "{raw_input.strip()}" is not a valid
number.')
          print("      Please use digits only, e.g. 42 or -3.5")
          continue
        if abs(number) > MAX_ALLOWED:
          print(f"  ⚠  That number is too large (limit:
±{MAX_ALLOWED:,}).")
          print("      Please enter a more reasonable value.")
          continue
        return number
    def check_even_or_odd(number):
      if not number.is_integer():
        return "Neither (decimal number)"
      if int(number) % 2 == 0:
        return "Even"
      else:
        return "Odd"
    def check_sign(number):
```

```
    if number > 0:
      return "Positive"
    elif number < 0:
      return "Negative"
    else:
      return "Zero"
  def display_results(number, even_odd, sign):
    display_number = int(number) if number.is_integer() else
number
    print("\n" + "=" * 42)
    print(f"   Results for: {display_number:,}")
    print("=" * 42)
    print(f"   ➤ Even or Odd : {even_odd}")
    print(f"   ➤ Sign        : {sign}")
    print("=" * 42)
    if number == 0:
      print("   ℹ  Zero is even but has no positive/negative
sign.")
      print("=" * 42)
  def main():
    print("=" * 42)
    print("       Welcome to Number Checker v2!")
    print("=" * 42)
    print("  Accepts whole numbers and decimals.")
    print("  Type 'q' at any time to quit.")
    while True:
      number = get_number_from_user()
      if number is None:
        print("\n  Goodbye! Thanks for using Number Checker.\n")
        break
      even_odd = check_even_or_odd(number)
      sign     = check_sign(number)
      display_results(number, even_odd, sign)
```

```
    if __name__ == "__main__":
      main()
```

Observation after improvement

Claude did not merely add invalid-input handling. The saved improved answer broadened the assignment into a much more advanced program: it added repeated execution, quitting with q, whitespace stripping, empty-input detection, decimal support, a size limit, and richer formatting.

<u>Gemini</u>

Initial response + improved response

Gemini gave a clear, conventional beginner answer. It used explicit comments, a variable called user_number, and separate printed messages for sign and parity.

Initial generated code from response

```
     # Ask the user to enter a number and store it in a variable
called 'user_number'
    # We use int() to convert the user's text input into an integer
(a whole number)
    user_number = int(input("Please enter a whole number: "))
    # --- Part 1: Check if it is positive, negative, or zero ---
    if user_number > 0:
      print(f"{user_number} is a positive number.")
    elif user_number < 0:
      print(f"{user_number} is a negative number.")
    else:
      print("The number is exactly zero.")
    # --- Part 2: Check if it is even or odd ---
    if user_number % 2 == 0:
      print(f"{user_number} is an even number.")
```

```
    else:
      print(f"{user_number} is an odd number.")
```

Brief description of the initial result

- The solution matched the task well for whole-number input and included comments plus a beginner explanation.
- The initial version assumed integer input directly and did not validate it.
- The naming style differed from the other tools by using user_number

Improved generated code from response

```
     # We use a 'while True' loop to keep asking for input until
the user gets it right.
    while True:
      try:
        # Ask the user for input
        user_input = input("Please enter a whole number: ")
        # Try to convert the text into an integer
        user_number = int(user_input)
        # If the conversion succeeds, we 'break' out of the loop
and move on
        break
      except ValueError:
        # If the user typed text or a decimal, int() will trigger a
ValueError.
        # We catch that error here, print a friendly message, and
the loop repeats.
        print("Oops! That is not a valid whole number. Please try
again.\n")
    # --- Part 1: Check if it is positive, negative, or zero ---
    if user_number > 0:
      print(f"{user_number} is a positive number.")
```

```
    elif user_number < 0:
      print(f"{user_number} is a negative number.")
    else:
      print("The number is exactly zero.")
    # --- Part 2: Check if it is even or odd ---
    if user_number % 2 == 0:
      print(f"{user_number} is an even number.")
    else:
      print(f"{user_number} is an odd number.")
```

Observation after improvement

Gemini answered the additional prompt directly and appropriately: it wrapped the original integer-based solution in a retry loop with try/except ValueError and explained the change clearly.

6. COMPARISON TABLE

5 - excellent, 4 - good, 3 - satisfactory, 2 - poor, 1 - very poor

Criterion	ChatGPT	Claude	Gemini
Understanding of the task	**5**	**5**	**5**
Completeness of the solution	**5**	**4**	**4**
Code correctness	**5**	**5**	**5**
Clarity of explanation	**5**	**5**	**4**
Usability	**4**	**3**	**4**
Program improvement	**5**	**3**	**5**
Total	**29 / 30**	**25 / 30**	**27 / 30**

7. ANALYSIS OF RESULTS

Which tool understood the task best?

All three tools understood the core task. ChatGPT and Gemini stayed closer to the exact assignment wording, while Claude correctly understood it but immediately expanded the scope beyond the initial requirement.

Which tool provided the highest-quality code?

For this specific beginner assignment, ChatGPT provided the highest-quality overall answer because it stayed close to the prompt, explained the code clearly, and improved the program without overcomplicating it. Claude produced the most advanced engineering solution, but it became less aligned with the narrow assignment after the second prompt.

Which response was the most understandable?

ChatGPT was the easiest response for a beginner to understand because the explanation was simple and the code remained compact. Gemini was also easy to follow, but narrower because it stayed with whole numbers. Claude was highly instructive, but its structure and added features made it more advanced.

Which tool was the most convenient to use?

Based on the responses, Gemini and ChatGPT were the most convenient to scan quickly, but ChatGPT remained slightly stronger overall because its explanation was more useful for learning. Claude was the least convenient in this comparison because its response was much longer and its improved version added many extra features beyond the requested change.

Can AI be fully trusted in programming?

No. AI tools are useful assistants, but generated code must always be checked for correctness, edge cases, input validation, and suitability for the real task. Even when the code looks correct, the programmer remains responsible for testing and verification.

8. CONCLUSION

This laboratory work showed that modern AI tools can solve simple programming tasks quickly and correctly when the prompt is clear.

The most useful tool for a beginner programmer in this comparison was ChatGPT, because it combined correctness, a clear explanation, and a direct improvement for invalid input. Claude was technically the most advanced and structured, but it expanded the assignment too far for a simple beginner exercise. Gemini gave a correct and beginner-friendly solution, but stayed more conservative and focused on whole-number input.

Answers to the required conclusion questions

1. **What are AI tools for programming?**

   They are intelligent assistants that can generate code, explain code, detect errors, and suggest improvements.

2. **What advantages do AI tools provide in coding?**

   They save time, help with routine tasks, give quick examples, explain program logic, and support learning.

3. **What are the disadvantages of AI-generated code?**

   The code may contain logical mistakes, unsafe assumptions, weak input validation, or explanations that are too general or incomplete.

4. **Why should generated code always be verified?**

   Because the programmer must make sure the program is correct, safe, and suitable for the actual task and test cases.

5. **Which of the tested tools was the most useful for a beginner programmer?**

   In this comparison, ChatGPT was the most useful because its saved answers were clear, correct, and well balanced between explanation and simplicity.

6. **Would you trust AI to write real code?**

   Only with human review, testing, and verification. AI is a strong assistant, but not a replacement for responsible programming practice.

Overall conclusion

AI tools are very helpful in education and in the early stages of software development. They can accelerate coding and help students understand program structure, but they differ in style, scope, and assumptions. The best approach is to treat AI as an assistant: useful for generating drafts and explanations, but always followed by human review, testing, and correction.